

Spring Boot Config — Quick Revision Notes

1. Why Configuration is Needed

App = Code + Configuration. Business logic is code, but things like port, DB URL, username change per environment — these are configuration values.

2. Problem with Hardcoding

Hardcoding values in Java code means rebuild+redploy for every change, code gets tied to one environment, and secrets can leak on GitHub.

3. application.properties

Spring Boot's default config file. It auto-loads on startup and stores values centrally instead of inside Java code.

4. Why named application.properties

Spring Boot automatically searches for this exact filename at startup and loads values from it if it exists.

5. Key-Value Format

Simple format: `key=value`. Example: `server.port=8081`.

6. @Value Annotation

Used to inject a config value into a Java variable. Example:

`@Value("${app.welcome.message}")`. This is called **property injection**.

7. Limitations of application.properties

Flat structure (no real hierarchy), repeated prefixes for grouped configs, and lists look messy as a single comma string.

8. Introduction to YAML

YAML solves the readability problem — same flat data written in a nested, structured way (`.yaml` file).

9. How Spring Boot Reads YAML

YAML looks nested but internally Spring Boot converts it to normal dot-separated keys (e.g. `server: port: 8081` → `server.port=8081`).

10. properties vs yaml

Both are supported (`application.properties`, `application.yaml`, `application.yml`) — but

stick to **one format** per project to avoid confusion.

11. YAML Syntax Rules

- Key followed by colon + space (`key: value`), not (`key:value`)
- Indentation (spaces only, no tabs) shows parent-child relationship
- Same-level keys need same indentation
- Comments use `#`
- Strings can have quotes or not

12-14. Converting properties → yaml

Same data, just written in nested YAML style — e.g. `spring.datasource.url=...` becomes grouped under `spring: datasource: url: ...`.

15-16. Lists in YAML

Use `-` for list items:

```
yaml

supported-cities:
  - Delhi
  - Mumbai
```

Lists of objects (multiple fields per item) are also possible — good use case for `@ConfigurationProperties`.

17-18. Reading Values (Dot Notation & Lists)

- Single value: `@Value("${app.name}")`
- List item by index: `@Value("${app.supported-cities[0]}")`
- Comma-separated string → List: `@Value("#{ '${...}'.split(',') }")`

19-21. Why Spring Profiles Are Needed

Same app runs in different environments (dev/test/staging/prod) needing different DB URLs etc. Manually commenting/uncommenting config is risky (could connect to prod DB by mistake).

22. What is a Profile

A profile = named environment mode (dev, test, staging, prod). It tells Spring Boot **which config to use right now**.

23-24. Profile-Specific Files

Naming pattern: `application-{profile}.properties` (or `.yaml`). Example: `application-dev.properties`, `application-prod.properties`, `application.properties` = common/default file for shared values.

25-26. Common File + Profile File Merge

Both common file and active profile file get loaded together. If same key exists in both, **profile-specific value wins** (overrides common).

27-31. Profile YAML Files

Same concept works with `.yaml`: `application-dev.yaml`, `application-prod.yaml` etc.

32-34. Activating a Profile

Need to explicitly tell Spring Boot which profile to use — multiple ways exist (see below). Same controller code gives different output based on active profile.

35. Production Note

Don't hardcode `spring.profiles.active=prod` inside the packaged JAR — pass it from outside so the same JAR works everywhere.

36. Activate via Command Line

```
java -jar app.jar --spring.profiles.active=dev
```

37. Activate via Maven

```
mvn spring-boot:run -Dspring-boot.run.profiles=dev
```

38-39. Activate via Environment Variable

```
export SPRING_PROFILES_ACTIVE=dev
```

Preferred in production (Docker, Kubernetes, CI/CD set this externally).

40-41. Practical Demo

Same JAR + different profile flag → different port/message (e.g. dev runs on 8081, prod on 9090). Only the active profile changes, code stays same.

42-43. @Profile Annotation — Why Needed

When you need **different bean implementations** per environment (not just different values) — e.g. fake email service in dev, real email service in prod. Without `@Profile`, Spring gets confused with multiple beans of same type.

44-45. @Profile Syntax

```
java

@Service
@Profile("dev")
public class DevNotificationService implements NotificationService { ... }
```

Only the bean matching the active profile gets created; the other is skipped.

46. Profile Files vs @Profile — Key Difference

- **Profile-specific files** → use when only **values** change (port, DB URL, messages)
- **@Profile** → use when the **class/bean implementation itself** changes (Dev vs Prod service)

47. Quick Comparison Table

Concept	Purpose
application.properties	simple key-value config
application.yml	structured/nested config
Profile-specific files	different values per environment
spring.profiles.active	tells which profile to use
@Value	read single value
@Profile	create bean only for a profile

48. Common Mistakes

- Hardcoding env values in Java code
- Keeping both `.properties` and `.yml` for same purpose
- Hardcoding `prod` profile inside JAR
- Overusing `@Value` for complex config (use `@ConfigurationProperties` instead)

- Confusing profile files with `@Profile`

49. Final Mental Model

Same Code + Same Build → Different Environment → Different Configuration.

- `.properties`/`.yml` → give config values
- Profiles → let same app run differently per environment
- `@Profile` → let Spring create different beans per environment